GRAND CHALLENGE - 2025

Shakti SDK User Manual



DEVELOPED BY :
SHAKTI DEVELOPMENT TEAM @ IITM
SHAKTI.ORG.IN

Proprietary Notice

Copyright © 2025, **SHAKTI@IIT Madras**.All rights reserved.
Information in this document is provided “as is,” with every effort ensuring that the documentation is as accurate as possible.

SHAKTI@IIT Madras expressly disclaims all warranties, representations, and conditions of any kind, whether express or implied, including, but not limited to, the implied warranties or conditions of merchantability, fitness for a particular purpose and non-infringement.

SHAKTI@IIT Madras does not assume any liability rising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation indirect, incidental, special, exemplary, or consequential damages.

SHAKTI@IIT Madras reserves the right to make changes without further notice to any products herein.

The project was funded by Ministry of Electronics and Information Technology (MeitY), Government of India

Release Information

Version	Date	Updates by	Reviewed by
1.0	16/09/2025	Nambirajan G, Jayanth R, Suganthi D	Kotteeswaran E
1.1	17/09/2025	Jayanth R	Kotteeswaran E
1.2	22/09/2025	Jayanth R	Kotteeswaran E
1.3	05/10/2025	Jayanth R	Kotteeswaran E
Draft V.1	07/10/2025	Jayanth R	Kotteeswaran E

Table of Contents

Proprietary Notice **02**
Release Information **03**
Table of Contents **04**
1. Introduction **05**
2. Software: **05**
2.1 SHAKTI-SDK 05
2.2 Pre-requisite 05
2.3 Application to output flow 06
3. Board Setup **07**
3.1 Powering the board 07
3.2 Programming SHAKTI 08
4. Software development flow **09**
4.1 SHAKTI-SDK Architecture 09
4.2 Board Support Package 10
4.3 Software 11
5. Setting up the GCSDK **13**
5.1 Makefile 13
5.2 Running Application 14
5.3 Application Development 20
5.4 Project Development 25
5.3 Make Upload Utility 28

1. Introduction

This document explains how the software Stack is built around Shakti Core. For the Grand Challenge 2025 Hackathon, there are two variants of Shakti C-Class namely Ganga SoC (64-Bit variant) and Yamuna(32-Bit Variant). This document details the SDK organisation, compilation of the code and steps to run the application for both SoCs in the SDK environment.

GC2025 SHAKTI SoC*:

1. GANGA SoC

Ganga is a 64-bit RISC-V SoC based on the SHAKTI C-Class core with a 6-stage pipeline, compliant with the RV64IMACSU instruction set.

2. YAMUNA SoC

Yamuna is a 32-bit RISC-V SoC based on the SHAKTI C-Class core with a 6-stage pipeline, compliant with the RV32IMAC instruction set.

*Note: To know more about the peripheral count and other details, refer to the Shakti SoC Register Manual.

2. SOFTWARE:

2.1 SHAKTI-SDK

Software Development Kits (SDKs) is an integral part of any product development. The main objective behind using a SDK is to reduce the development time as it is having all the necessary drivers, header files (SoC specific definitions, memory initialisation, core init, trap handling, debug log levels, etc.). The SHAKTI-SDK is a platform that enables developing applications over the SHAKTI processors. Firmware support is provided for the end users to develop applications. The SHAKTI-SDK is simple and easily customizable.

2.2 Pre-requisite

Before starting to develop any application make sure the following prerequisites have been met.

1. The Operating System being used is Ubuntu 22.04 (Recommended) or higher.
2. Installation of the pre-required tools are completed, if the tools are not yet installed refer to the “Tools Setup Manual” to complete the installation of required tools.

2.3 Application to Output Flow

After installing the prerequisites, it is important to understand the flow of how the binary is executed after compilation. Tools like OpenOCD and riscv-gnu-gdb helps in executing the compiled binary.

1. Openocd

OpenOCD stands for Open On-Chip Debugger. It's a tool that lets the host interact with target hardware (**figure 1**). It is like a bridge between host and target hardware. OpenOCD acts as server and listening for clients (gdb)



figure 1 : Openocd connection

2. GDB

GDB stands for GNU Debugger. It is a debugging tool designed for RISC-V architecture. It's a program that helps the user see what the code is doing inside the processor (**figure 2**) - while it runs, stops, crashes, or behaves weirdly. The gdb acts as client to openOCD and requests for client server connectivity. Once connectivity is established, the user can access core and peripheral mapped registers and can run and debug programs.

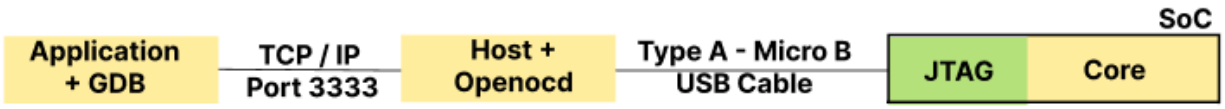


figure 2 : Openocd + GDB connection

3. BOARD SETUP

The following steps are needed to setup the board:

1. Power ON the board / Connect the board to the PC.
2. Program the BitStream (Ganga/Yamuna/) to the board.Refer this document to generate Bitstream and Program the FPGA

3.1 Power ON the board

Plug USB Type A connector end of cable into the PC's slot and the Micro B type connector end to connector (J10) in the board. This will power ON the board. please see **Figure 3 and Figure 4**. Check the Power LED to glow once the board is powered up. In addition to power up the board, the connector J10 is also used

- To program the FPGA.
- Acts as ftdi based jtag debugger used to access the Shakti debugger.
- Used as UART to view the console prints for debugging.

If external connected circuits requires more power, an external 12V power supply can be connected via Power Jack (J12) to fulfill the additional power requirements. Refer Arty reference manual for detailed power on instructions.

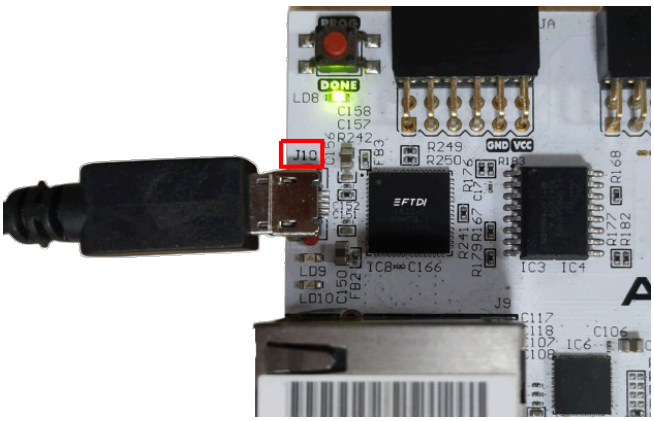


figure 3: Powering the Board

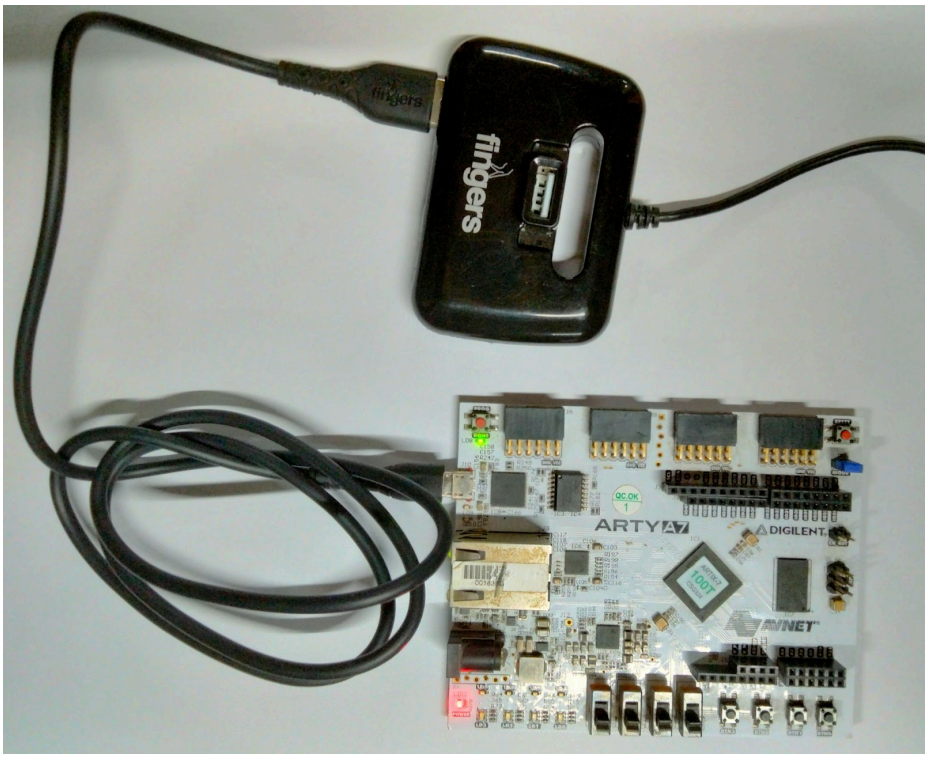


figure 4 : FPGA Board - PC Connection

```
rj@rj:~$ lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 096: ID 3554:fc03 CX 2.4G Receiver
Bus 001 Device 099: ID 0403:6010 Future Technology Devices International, Ltd FT2232C/D/H Dual UART/FIFO IC
Bus 001 Device 003: ID 05e3:0608 Genesys Logic, Inc. Hub
Bus 001 Device 005: ID 048d:5702 Integrated Technology Express, Inc. ITE Device
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
rj@rj:~$
```

figure 5 : lsusb command showing FPGA connection to PC

3.2 Programming SHAKTI

Before programming the FPGA, make sure the bitstream has been generated/available. Refer the “Bitstream/mcs generation manual” to generate bitstream for the target SoC. Once the bitstream/mcs is programmed, the core and peripherals of the SoC can be accessed and the program can be loaded and run. The following sections covers the overall organisation and the procedure to compile, load and run in openOCD - gdb environment with Shakti SDK.

4. SOFTWARE DEVELOPMENT FLOW

This section presents the software framework for design and implementation of embedded/IoT applications. This section will explain in detail, on how to develop applications using the SHAKTI Software Development Kit (SHAKTI-SDK).

4.1 SHAKTI-SDK Architecture

The SHAKTI-SDK is a C/C++ platform to develop applications for SHAKTI. The SDK has the necessary firmware code and framework to develop newer applications on the hardware. The framework is lightweight and customizable. **Figure 5** shows a picture representation of the framework

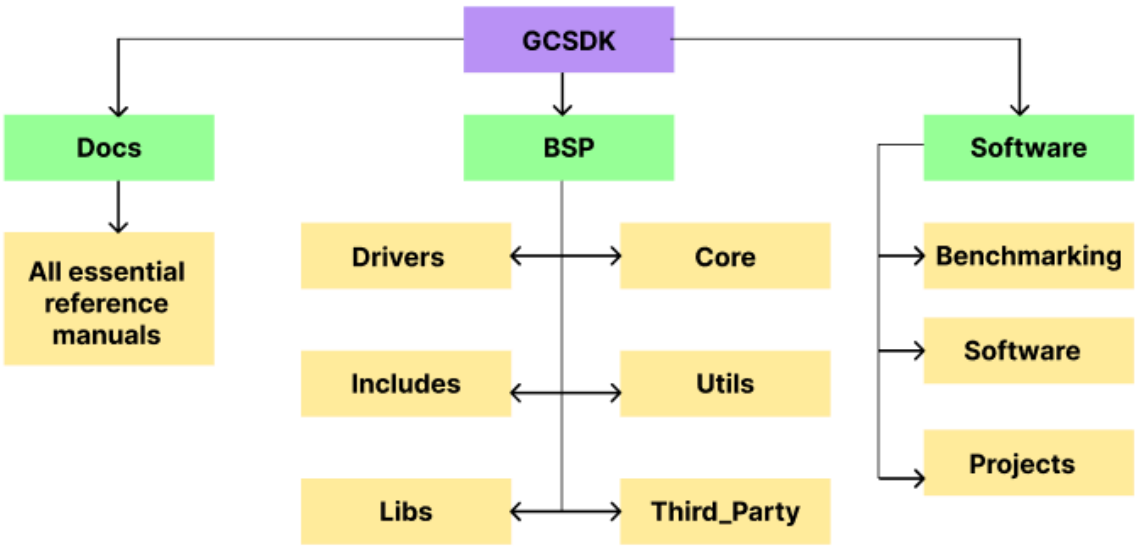


figure 6: Framework of GCSDK

4.2 Board Support Package

The BSP consists of driver files for various devices and system files. It contains certain platform dependent definitions for each board. Essentially, the BSP is the layer above the hardware. It includes the following sub directories.

1. drivers

The drivers are a set of software constructs that help software applications to access the devices/peripherals in the SoC. They are generally low level API's that execute a particular task in the hardware.

2. include

This folder has header files for core and each peripheral driver. The board independent variable/macro definitions and declarations pertaining to each driver is included here.

3. libs

The library utilities, boot code are hosted here. The library is a common place for reusable code. The libraries can be compiled as a separate "lib" file and used.

4. core

The core usually has functions related to the startup codes, Trap handlers and interrupt vectors. The codes related to memory initialisation are also available here.

5. utils

This contains the code related to the standalone mode feature of the SHAKTI processor.

6. third_party

This folder provides support for external boards as well as custom boards. Includes the definitions of board specific functions such as peripheral base address mapping, debugger configuration details, Addresses of different Memory segments for linking, etc. .

4.3 Software

The **“software”** folder provides a platform for developing various applications, independent of the underlying BSP. All the applications/projects developed in GCSDK reside in this folder. In general, an application will involve writing high level C/C++ code that uses BSP API’s. The software folder is broadly classified into the following sub-directories.

4.3.1 Examples

This is the place where any new standalone application is developed i.e. This folder contains the simple example application codes based on the single driver code. Few example programs involving sensors are already developed for different peripherals and kept here. These programs demonstrate the integration of BSP and the core support libraries with the user programs.

Inside the example directory there are multiple folders (**figure 7**) based on different protocols (e.g. UART, SSPI, I2C, PWM, etc..) with each folder containing multiple sample applications using the protocol mentioned in the parent folder name (**figure8**).

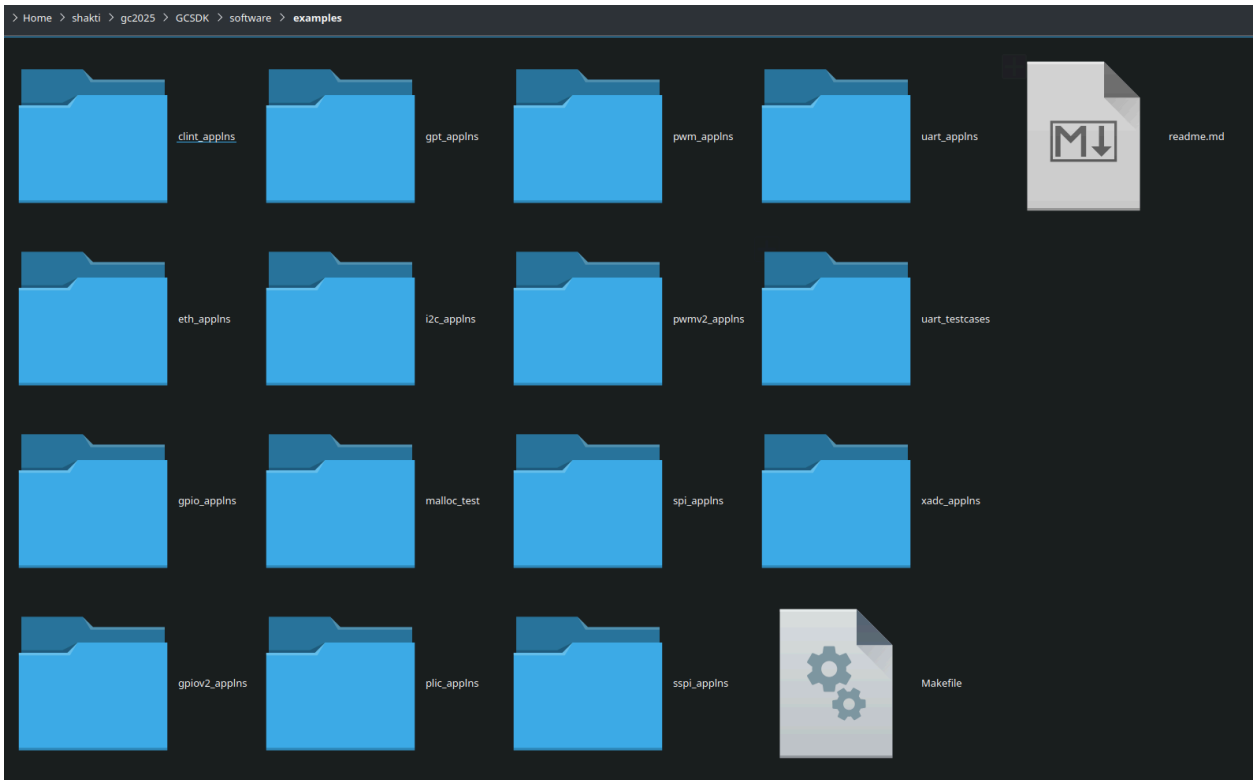


figure 7 : Folder of different protocols inside the example directory

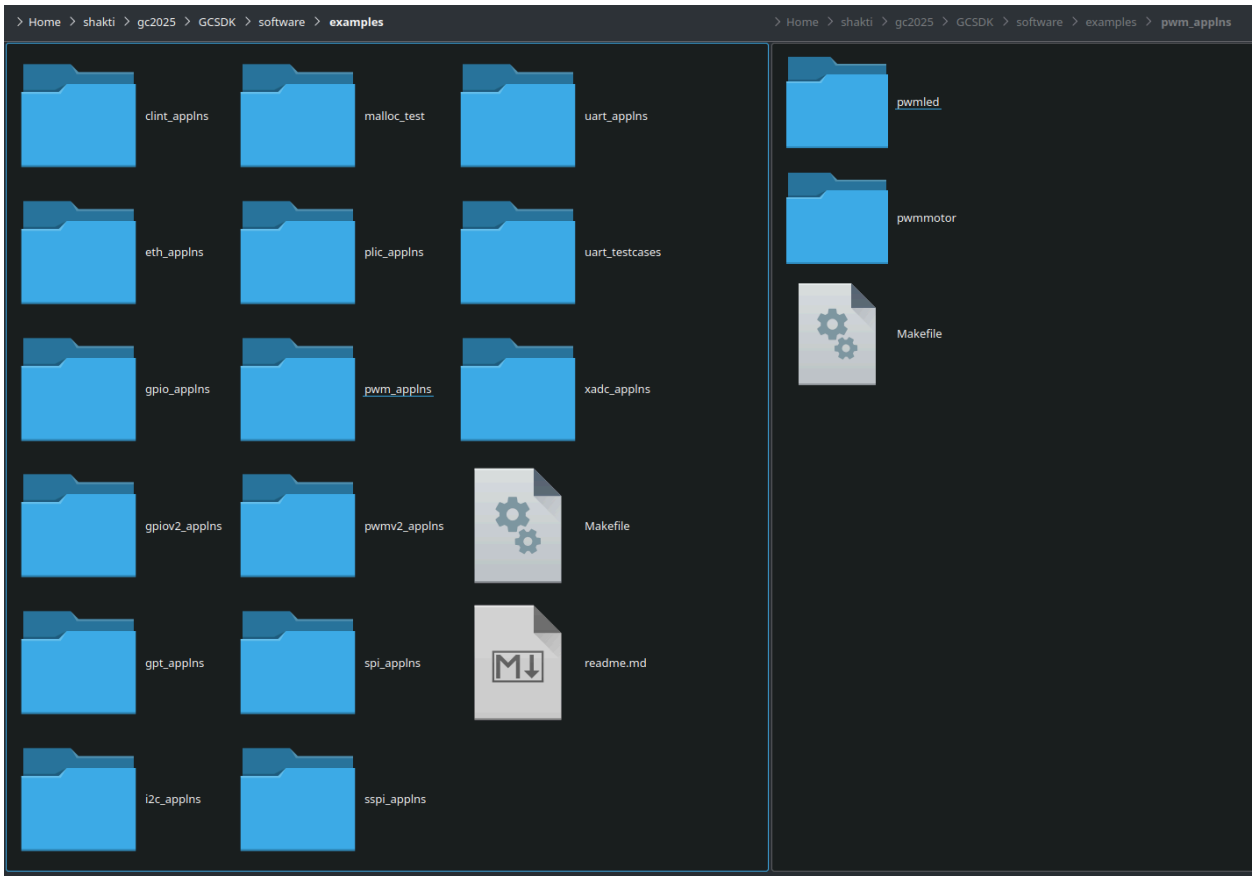


figure 8 : Different applications inside the folder pwm_appln

4.3.2 Projects

This is the place where any new integrated application with multiple sensors and protocols is developed i.e. application code as a project with multiple drivers. These programs demonstrate the integration of multiple drivers and libraries.

4.3.3 Benchmarking

Applications or bare metal codes that are developed for bench-marking a core reside here. These programs usually describe the capability of the SHAKTI class of Processors.

5. Setting up GCSDK

The GC2025 repository has a GCSDK folder which contains scripts, board support packages to build the application. It can be cloned by running the following command:

```
git clone https://gitlab.com/shaktiproject/gc2025.git
```

5.1 Makefile

The SDK uses Makefile based build utility in the terminal i.e. to compile programs more efficiently, the GNU's MAKE utility is used. The make utility uses the Makefile to compile programs from source code.The Makefile has support for different target boards and applications. The list of make commands can be found by typing ***make help*** in the terminal as shown in **figure 9**.

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ make help
 Shakti Software Development Kit
 Makefile features:

 list_targets
 lists the boards supported

 list_applns
 lists the example applns availabel to user

 software [PROGRAM=] [TARGET=pinaka]
 Builds the requested PROGRAM for the given TARGET

 debug [PROGRAM=] [TARGET=pinaka]
 Builds the requested PROGRAM with debug options

 upload [PROGRAM=] [TARGET=pinaka]
 Uploads the requested PROGRAM to the Flash memory in the board

 flash [PROGRAM=] [TARGET=pinaka]
 Flashes the requested PROGRAM to the Flash memory in the board

 clean
 Cleans compiled objects in every example applns.

 clean [PROGRAM=]
 Cleans compiled objects of that particular appln.
```

figure 9: Makefile features

5.2 Compilation and Running Application

1. Build

The makefile based build approach does the following

- Creates a folder with the name “output” in the application folder.
- Compiles all the drivers and integrates them as a single library.
- Compilation by application code
- Link both the integrated library and application code & generates executable with .shakti extension.
- Generates the dump file for debugging.

The compilation (build) steps are

1. Go to the GCSDK folder.
2. Run the make command to build the application code.

The following commands are used for building the application.

```
$ cd GCSDK
$ make software PROGRAM=hello TARGET=<Target_SoC>
```

Note: “Target_SoC” can either ganga or yamuna

2. Run

- Once the application is compiled, the executable is generated in the output folder. The executable is in ELF file format and has the extension “.shakti”.
- The Arty100T board should be connected to the OpenOCD debugger, in order to debug the program using the RISC-V GNU Debugger (GDB) software.
- After the application is compiled, the application code with “.shakti” extension can be loaded into the Arty 100T board and tested.

The following steps list out the actions to be taken.

As already given in the openOCD details the following steps can be followed to load the code into memory and execute the code.

- The host machine and the SoC needs to be connected through openOCD and gdb.
- Load the code into memory.
- Start running the code.
- See the console print in the UART terminal display.

The following sections explain the above steps in detail.

2.1 Steps to run

Open four tabs on terminal window, one for each of the following

A. Make window - to compile the code
B. OpenOCD - to run the openocd server.
C. GDB - run risc-v gdb.
D. UART terminal display (gtkterm) - console prints

Note :

- Please ensure that the working directory is GCSDK for all of above terminal tabs
- <Target SoC> can either be ‘ganga’ or ‘yamuna’

A.make window

The following command can be used to compile the application.

make software PROGRAM=hello TARGET=<Target_SoC>

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ make software PROGRAM=hello TARGET=yamuna
```

figure 10 : Command to compile “Hello” Program with yamuna as the Target SoC

Make sure the application code is compiled without any errors. **“All done” (figure 11)** indicates that the code has been successfully compiled without any errors. If there is any syntax error during compilation, the error is displayed in red color with the error details.

```
make[2]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples'
cd uart_applns/hello && make hello.riscv TARGET=yamuna DEBUG=
make[2]: Entering directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples/
/home/rj/belsoc/bsoc_toolchain/build__rv32imac/bin/../lib/gcc/riscv32-unknown-elf/13.2.0/../
known-elf/bin/ld: warning: ./output/hello.shakti has a LOAD segment with RWX permissions
make[2]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples/u
All done !
make[1]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples'
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$
```

figure 11 : Successful compilation of Hello Program

B.Openocd window

In the second terminal, launch OpenOCD (**figure 12**) with super user (sudo) permission. Press reset in the board and run the following commands.

```
cd bsp/third_party/<Target_SoC>
sudo ./openocd -f ftdi.cfg
```

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ cd bsp/third_party/yamuna/
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK/bsp/third_party/yamuna$ ls
bootlink.ld  ftdi.cfg  link.ld  openocd  platform.h
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK/bsp/third_party/yamuna$ sudo ./openocd -f ftdi.cfg
[sudo] password for rj:
Open On-Chip Debugger 0.12.0+dev-03694-gca7d88252 (2024-02-29-16:07)
Licensed under GNU GPL v2
For bug reports, read
        http://openocd.org/doc/doxygen/bugs.html
DEPRECATED! use 'adapter speed' not 'adapter_khz'
DEPRECATED! use 'adapter driver' not 'interface'
DEPRECATED! use 'ftdi vid_pid' not 'ftdi_vid_pid'
DEPRECATED! use 'ftdi channel' not 'ftdi_channel'
DEPRECATED! use 'ftdi layout_init' not 'ftdi_layout_init'
Info : auto-selecting first available session transport "jtag". To override use 'transport select <transport>'.
Info : Simple Register based Bscan Tunnel Selected
Info : clock speed 1000 kHz
Info : JTAG tap: riscv.cpu tap/device found: 0x13631093 (mfg: 0x049 (Xilinx), part: 0x3631, ver: 0x1)
Info : [riscv.cpu.0] datacount=12 progbufsize=16
Info : [riscv.cpu.0] Disabling abstract command reads from CSRs.
Info : [riscv.cpu.0] Disabling abstract command writes to CSRs.
Info : [riscv.cpu.0] Examined RISC-V core
Info : [riscv.cpu.0]  XLEN=32, misa=0x40001105
[riscv.cpu.0] Target successfully examined.
Info : [riscv.cpu.0] Examination succeed
Info : starting gdb server for riscv.cpu.0 on 3333
Info : Listening on port 3333 for gdb connections
riscv.cpu.0 halted due to debug-request.
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
```

figure 12 : Openocd connection from host to yamuna SoC

C. GDB window

In the third terminal launch RISC-V GDB. Applications will be loaded and run in this terminal.

```
riscv(XLEN)-unknown-elf-gdb
(gdb) set remotetimeout unlimited
(gdb) target remote localhost:3333
(gdb) file path/to/executable of the application
(gdb) load
(gdb) c
```

NOTE:

The file “software/program/uart_applications/hello/hello.shakti” is the program chosen is hello

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ riscv32-unknown-elf-gdb
GNU gdb (GDB) 14.2
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-pc-linux-gnu --target=riscv32-unknown-elf".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb)
```

figure 13 : invoking riscv(xlen)-gdb

```
(gdb) set remotetimeout unlimited
(gdb) target remote localhost:3333
Remote debugging using localhost:3333
warning: No executable has been specified and target does not support
determining executable automatically.  Try using the "file" command.
0x00001540 in ?? ()
(gdb) file software/examples/uart_applns/hello/output/hello.shakti
```

figure 14 : executing the commands mentioned above

```
Reading symbols from software/examples/uart_applns/hello/output/hello.shakti...
(gdb) load
Loading section .text.init, size 0xa0 lma 0x80000000
Loading section .text, size 0x4036 lma 0x800000a0
Loading section .text.strcpy, size 0x78 lma 0x800040d6
Loading section .rodata, size 0xab8 lma 0x80004150
Loading section .eh_frame, size 0x3c lma 0x80004c08
Loading section .sdata, size 0x8 lma 0x80004c44
Start address 0x80000000, load size 19530
Transfer rate: 48 KB/sec, 2790 bytes/write.
(gdb) c
Continuing.
```

Figure 15 : loading the “.shakti “ file and running it

D. GTKterm/ window

To open a serial port to display output from UART using the below command (**figure 16**) in the first terminal. It is assumed that “gtkterm” is already installed as a part of tools setup.

```
$ sudo gtkterm
```

NOTE:

Choose 'Click CRLF Auto' (**figure 17**)in the configuration menu , connect the USB port to which the FPGA is connected (**figure 18**) and the baud rate is set to 19200.(**figure 19**)

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ sudo gtkterm
```

figure 16 : Command to open gtkterm

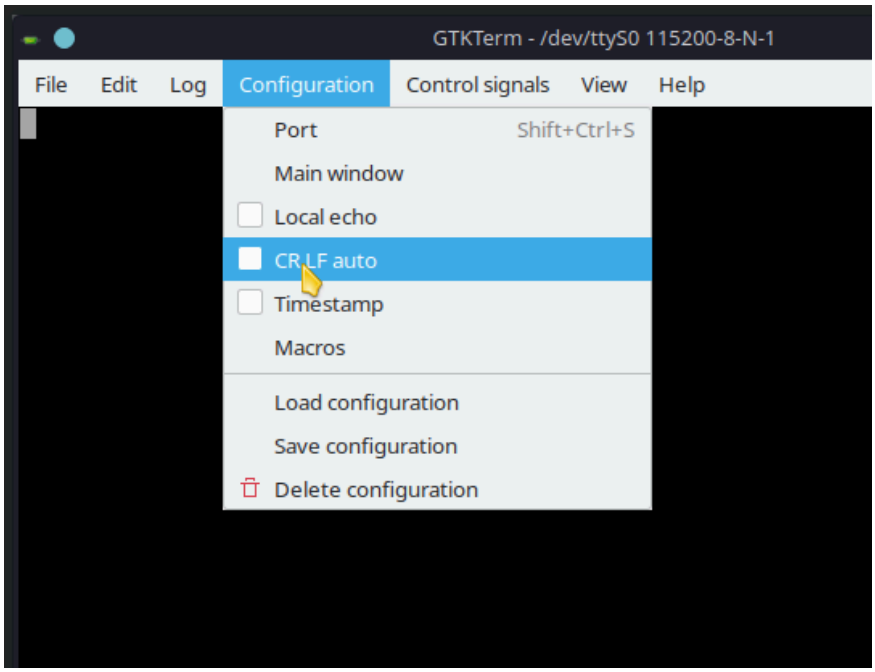


figure 17 : gtkterm , selecting CR LF auto Option

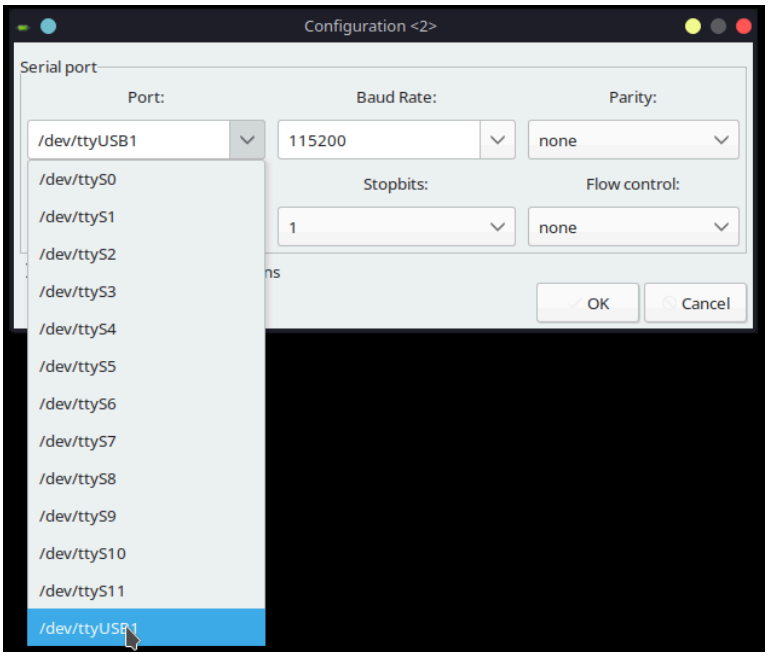


figure 18 : USB Port selection

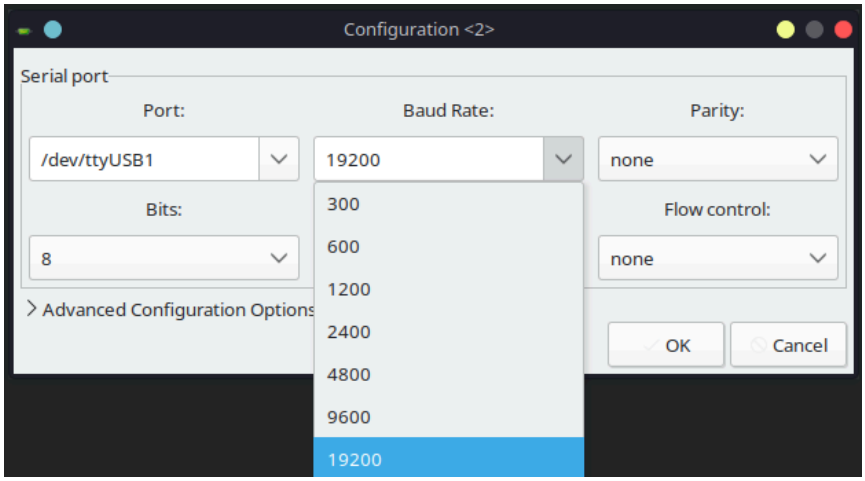


figure 19: baud rate selection

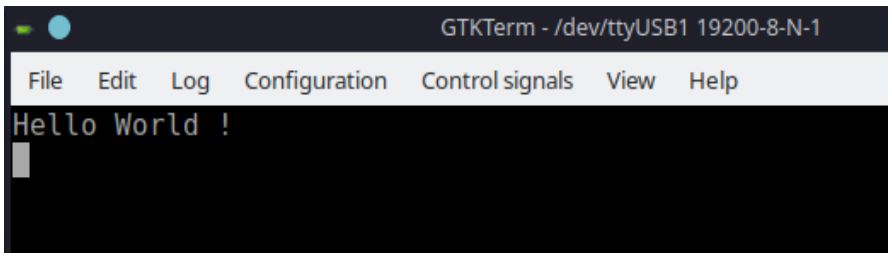


figure 20 : Display of output in gtkterm

5.3 Adding new Application Code in examples folder

GCSDK helps in developing applications for the Target SoC. Following are the steps to develop a new application using GCSDK.

Steps to add a new application to GCSDK

The steps followed to develop a simple program is listed below

- The new application is created under one of the examples/XYZ_applns[1] folder.
- Go to folder examples/XYZ_applns.
- Create a folder[2] for the new application and name it accordingly

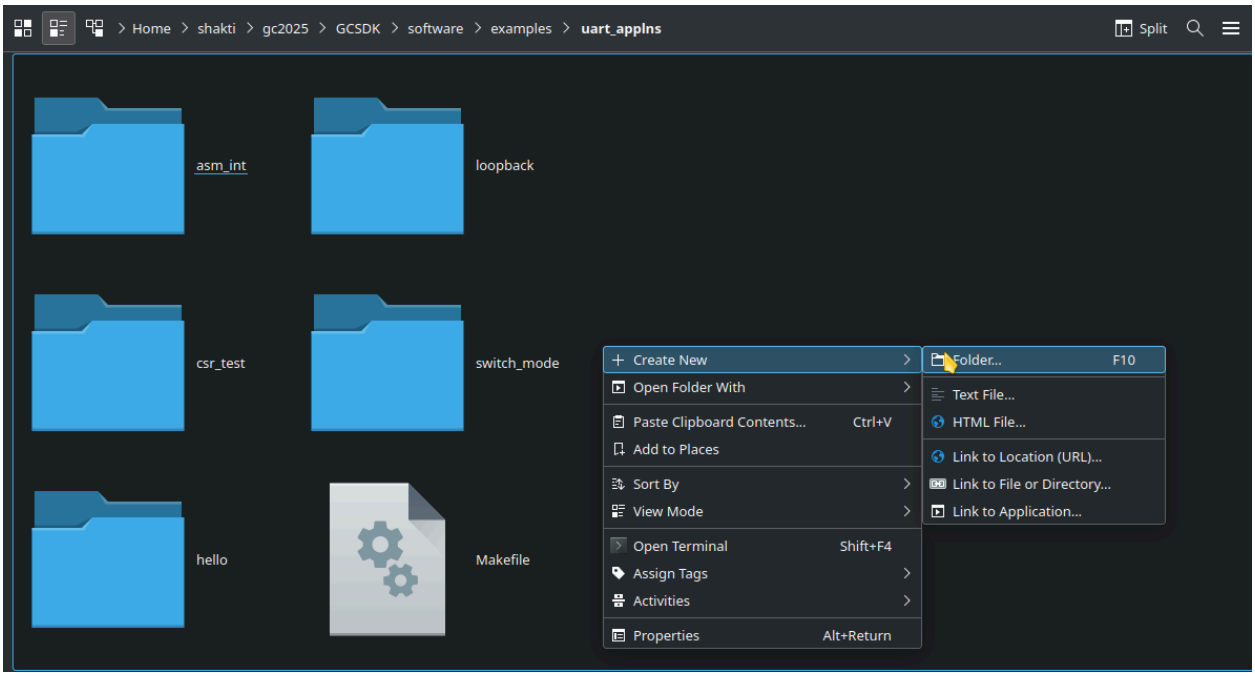


figure 21 : creating a new folder inside uart_applns

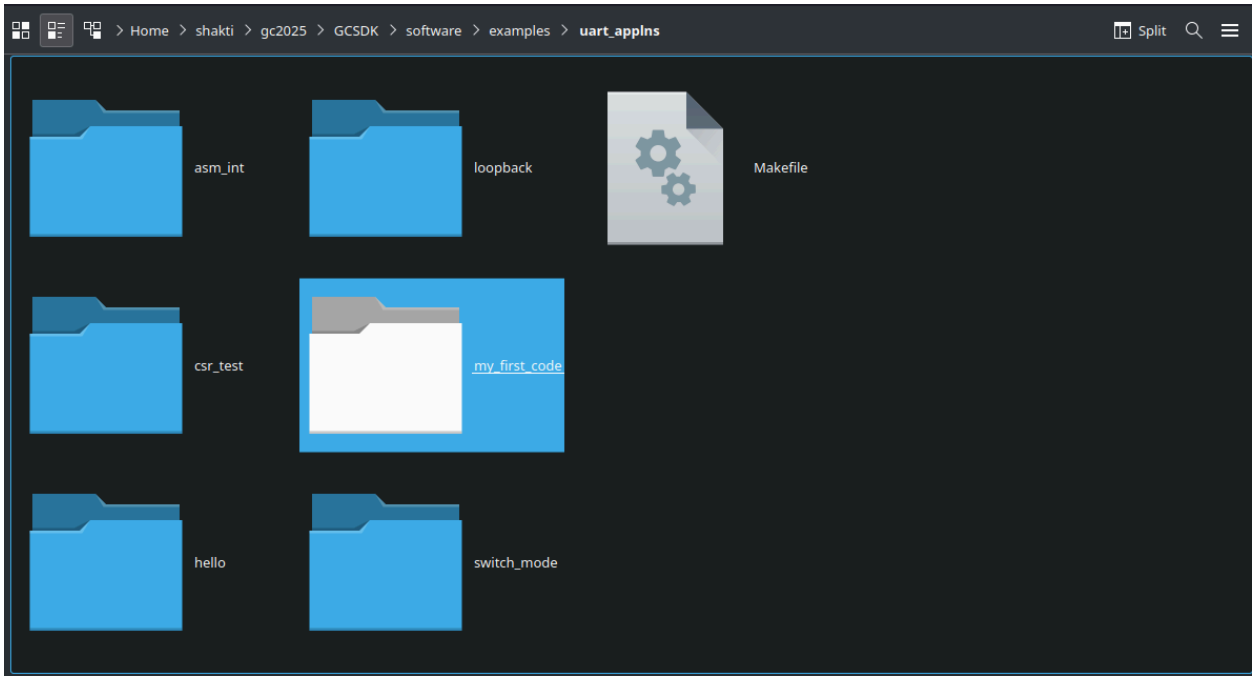


figure 22 : Naming the folder accordingly

- Inside the folder, create source file[2] for the application.



figure 23 : creating a new c file , named it same as the folder

- Add the necessary functions in the source files and call them from main.
- Add a makefile to the newly created folder
    a) Re-use existing Makefile file from another folder within the same application
    b) Create custom Makefile.

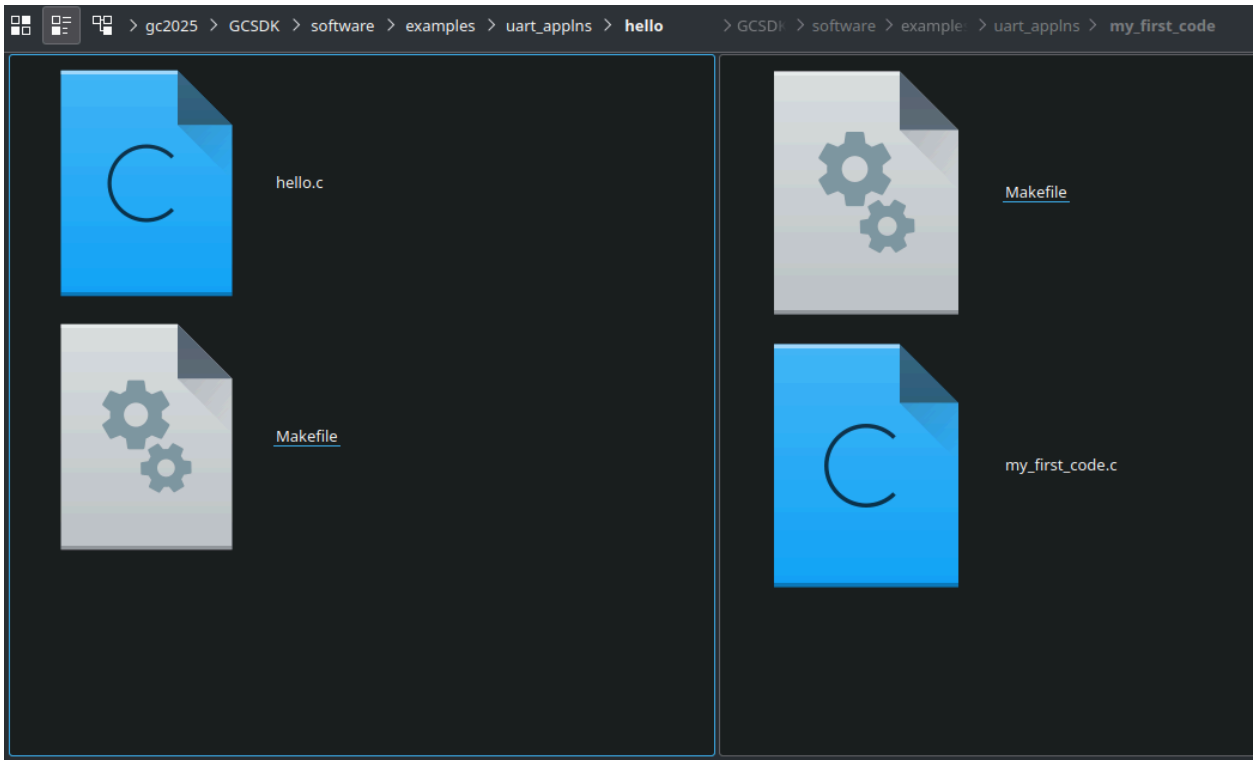


figure 24 : copied & pasted a makefile from different program inside the same application folder
In this example, Makefile is reused from the folder “Hello” inside “uart_applns” to “my_first_code”

- Make an entry in the existing Makefile under /GCSDK/software/examples for the new application.

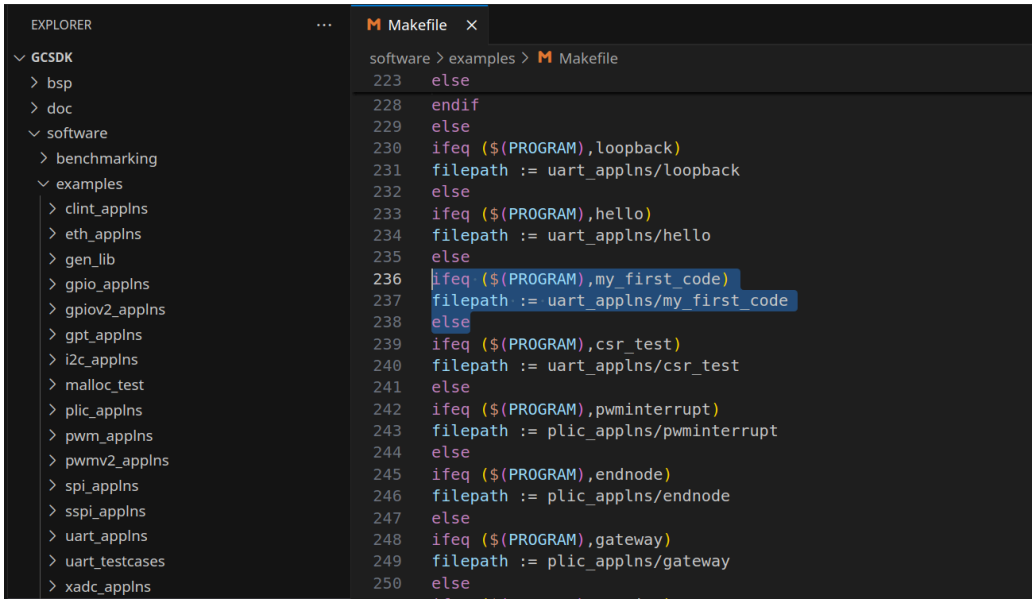


figure 25 : Adding an “ifeq” block along with application path

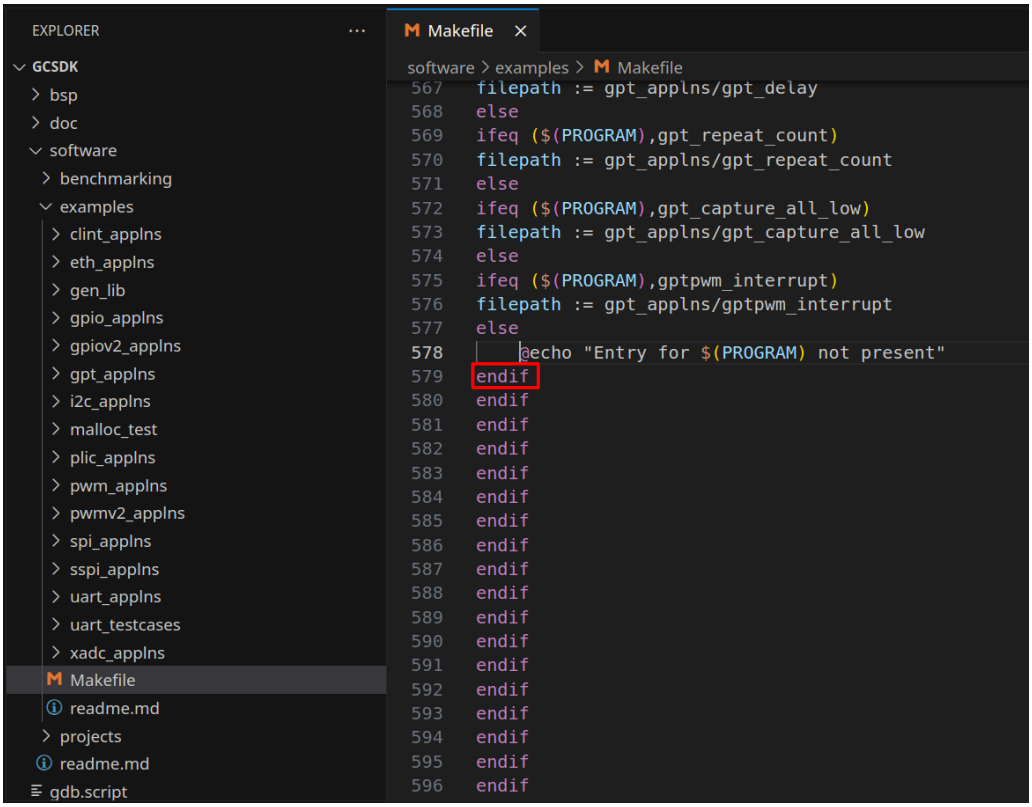


figure 26: Adding “endif” block to terminate “ifeq” block

- Follow steps in section 5.2 to compile and run the application created .

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ make software PROGRAM=my_first_code TARGET=yamuna
```

figure 27 : compiling “my_first_code”

```
make[2]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples'
cd uart_applns/my_first_code && make my_first_code.riscv TARGET=yamuna DEBUG=
make[2]: Entering directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples/u
/home/rj/belsoc/bsoc_toolchain/build__rv32imac/bin/../lib/gcc/riscv32-unknown-elf/13.2.0/../.
ode.shakti has a LOAD segment with RWX permissions
make[2]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples/ua
All done !
make[1]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/examples'
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$
```

figure 28 : Successful compilation of “my_first_code”

NOTE :

[1] : XYZ_applns is the name of the driver for which applications need to be added. For e.g. if the UART driver is being used, the application code can be added to the “uart_applns” folder.

[2] :folder name and application file name should be the same

5.4 Adding new Project in Projects folder

Steps to add a new project to GCSDK

The steps followed to develop a simple project is listed below

- Inside the projects directory , create a new folder for the project and name it

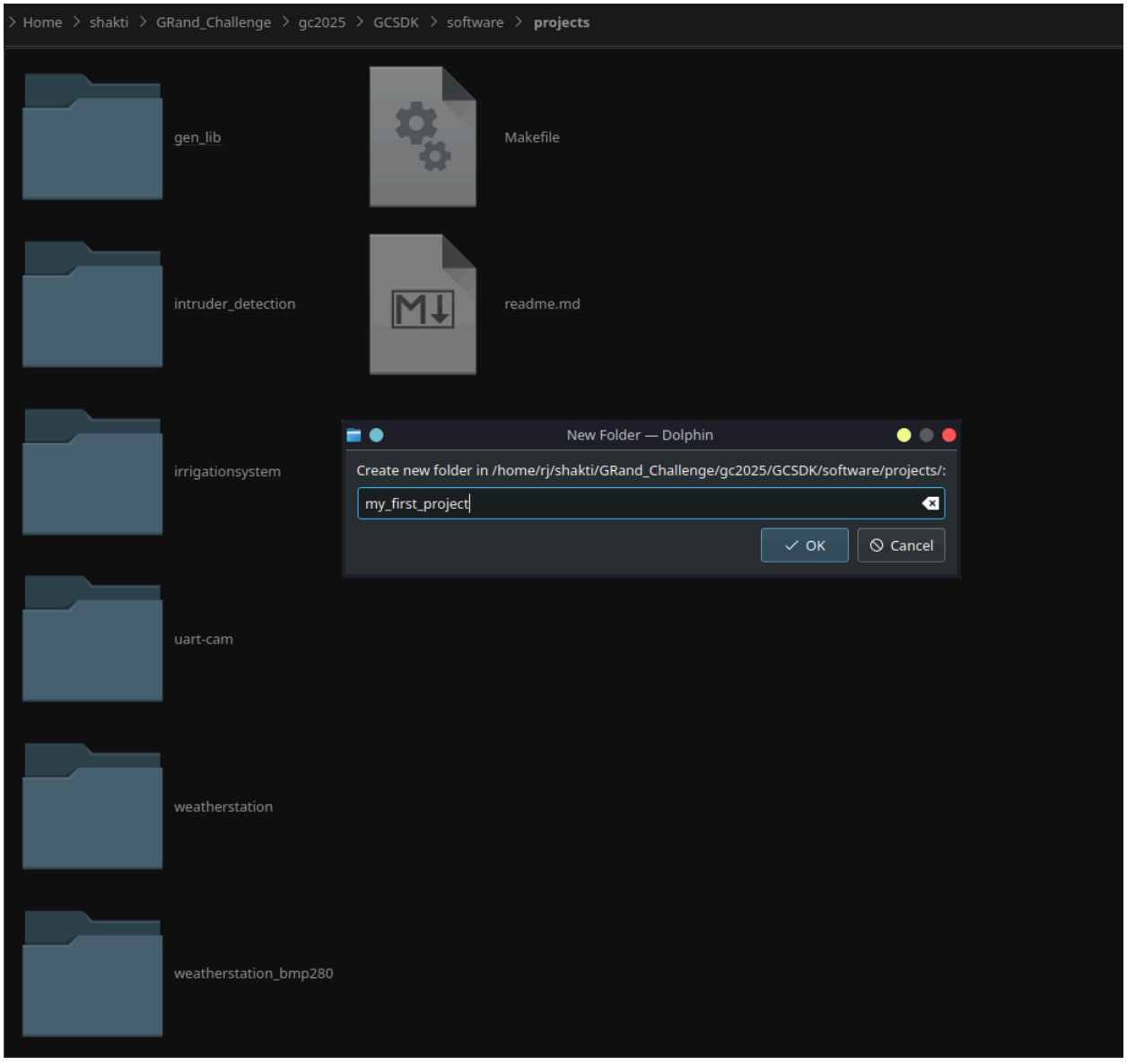


figure 29 : Creating a new folder for new project named “my_first_project”

- Inside the newly created folder , create the application with the same name as the folder
- Reuse / Create a makefile inside the newly created folder

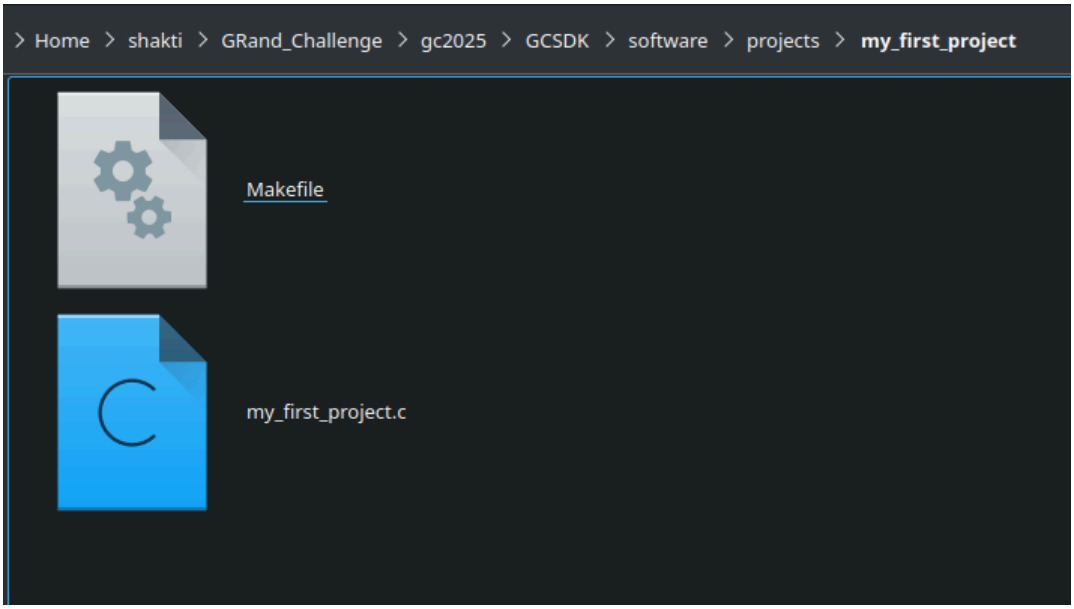


figure 30 : Added application with same name as the folder and makefile , in this case , the makefile has been reused from already existing project

- Add “ifeq” block along with application path and terminate it with “endif” block in the makefile in projects directory

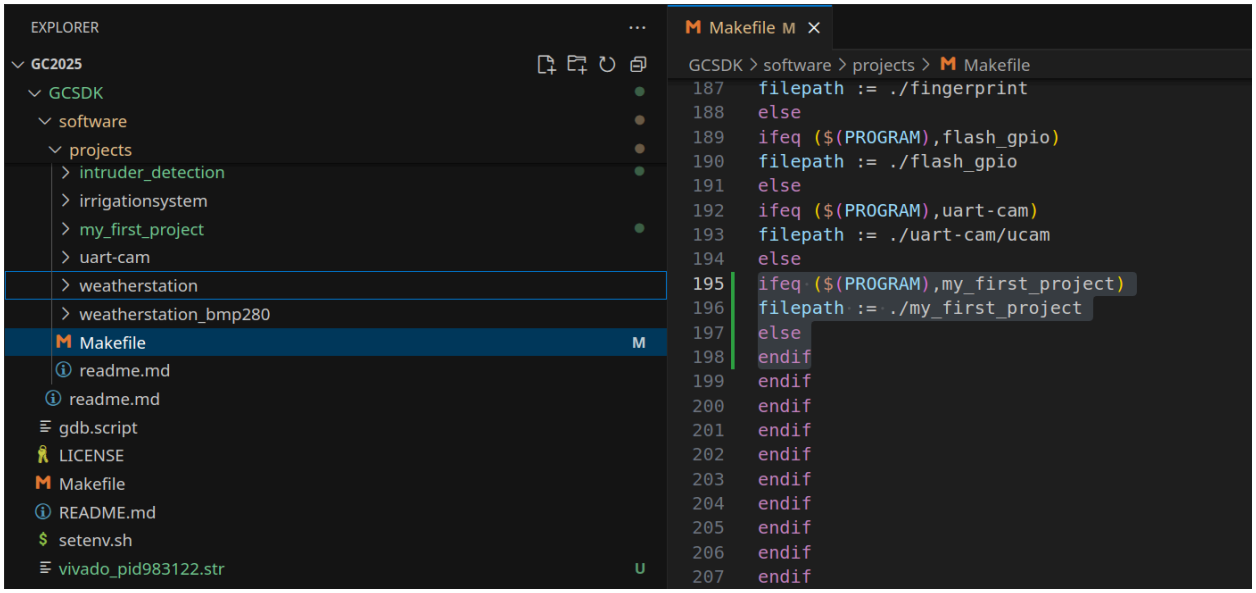


figure 31 : Added “ifeq” , path to project and “endif” block in makefile in projects directory

- Compile the newly created project using the following command

make project PROGRAM=my_first_project TARGET=<Target_SoC>

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ make project PROGRAM=my_first_project TARGET=yamuna
```

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ make project PROGRAM=my_first_project TARGET=yamuna
Build my_first_project on yamuna board
cd ./software/projects && make PROGRAM=my_first_project TARGET=yamuna
make[1]: Entering directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/projects'
make compile build
make[2]: Entering directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/projects'
Compiling /home/rj/shakti/GRand_Challenge/gc2025/GCSDK/bsp/drivers
/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/bsp/third_party/yamuna
/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/bsp/drivers
make[2]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/projects'
cd ./my_first_project && make my_first_project.riscv TARGET=yamuna DEBUG=
make[2]: Entering directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/projects/my_first_proj
ect'
/home/rj/belsoc/bsoc_toolchain/build__rv32imac/bin/../lib/gcc/riscv32-unknown-elf/13.2.0/../../../../risc
v32-unknown-elf/bin/ld: warning: ./output/my_first_project.shakti has a LOAD segment with RWX permissions
make[2]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/projects/my_first_proje
ct'
TARGET is yamuna
make[1]: Leaving directory '/home/rj/shakti/GRand_Challenge/gc2025/GCSDK/software/projects'
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$
```

figure 32 : Command to compile new created project

- Connect to openocd and riscv-gdb to execute the project as done in **section 2.1 steps B,C and D**

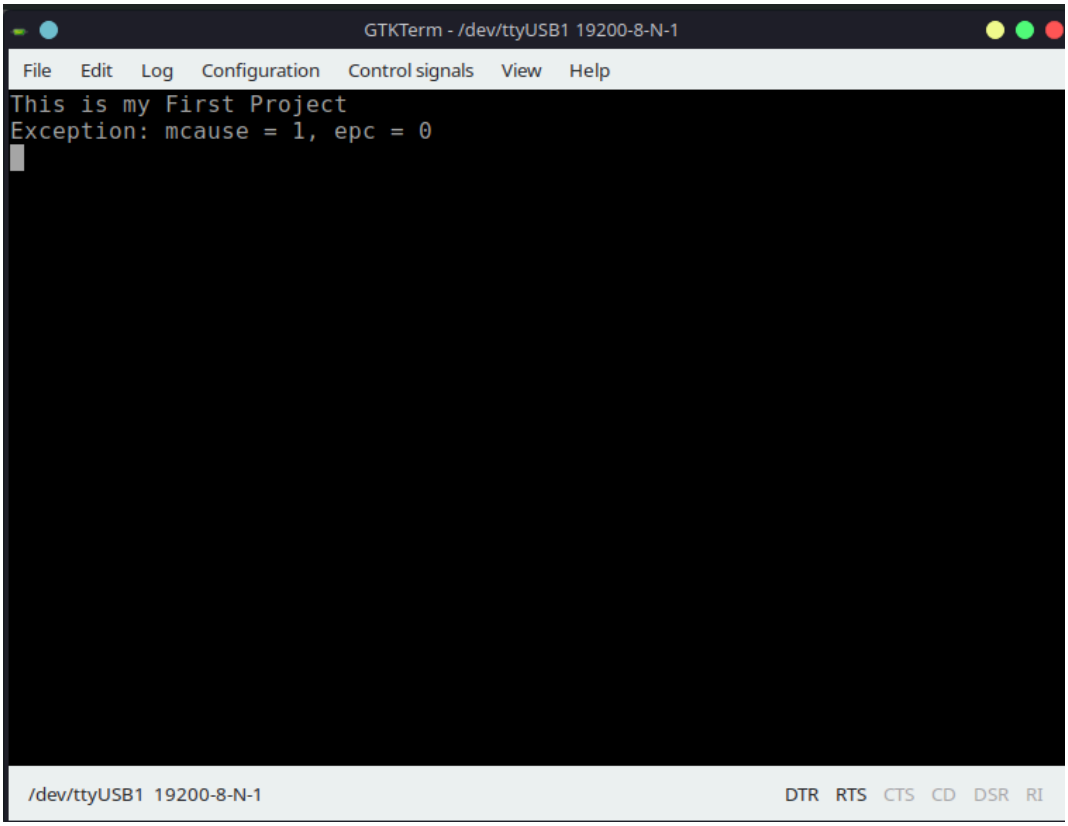


figure 33 : Output console of “my_first_project” program

5.5 Using make upload utility

Make upload is a special utility part of SJAKTI , which allows the user to compile and burn the application into the FPGA, once make upload is done , there is no need to connect to openocd and riscv-gnu-gdb to execute the application

Steps to use make upload utility

- Go to /GCSDK directory in the tab1 of the terminal
- In the 2nd tab of the terminal open gtkterm and select correct usb port and set the appropriate baud rate , refer to **section 2.1 step D**
- In the 1st tab run the following command

```
make upload PROGRAM=hello TARGET=<Target_SoC>
```

```
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$ make upload PROGRAM=hello TARGET=yamuna
```

figure 34 : make upload command

- After entering the pwd , openocd will automatically connect , upload the code and terminate. An “All Done” message will indicate the same.

```
All done !
make[1]: Leaving directory '/home/rj/shakti/GRand_C
hallenge/gc2025/GCSDK/software/examples'
rj@rj:~/shakti/GRand_Challenge/gc2025/GCSDK$
```

figure 35 : “All done” message indicating upload is completed

- Simultaneously in the gtkterm , console log will be visible that flash is erased and then the flash is written again.

```
File  Edit  Log  Configuration  Control signals  View  Help
sspi_instance[0]: 20000sspi_instance[1]: 20100SPI init done
command [35] read value: 0
command [5] read value: 2

 data[0]: 1
 data[1]: 20
 data[2]: 18
 data[3]: 4d
 data[4]: 1
 data[5]: 80
 Device id is 12018
 Flash device id read complete
 Flash Erase command is sent
 Flash Erase command is sent
Erase complete
 Data read after erase:
 Flash read complete.............

Write complete.

Please reset.
```

figure 36 : console log in gtkterm indicating flash write is done

- Press the reset button on the FPGA board.

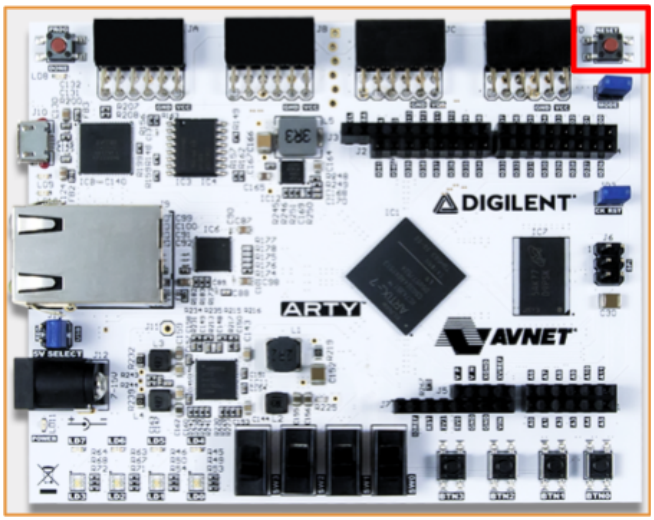


figure 37 : red box indicating the reset button

- After pressing the reset button the output will be visible in gtkterm

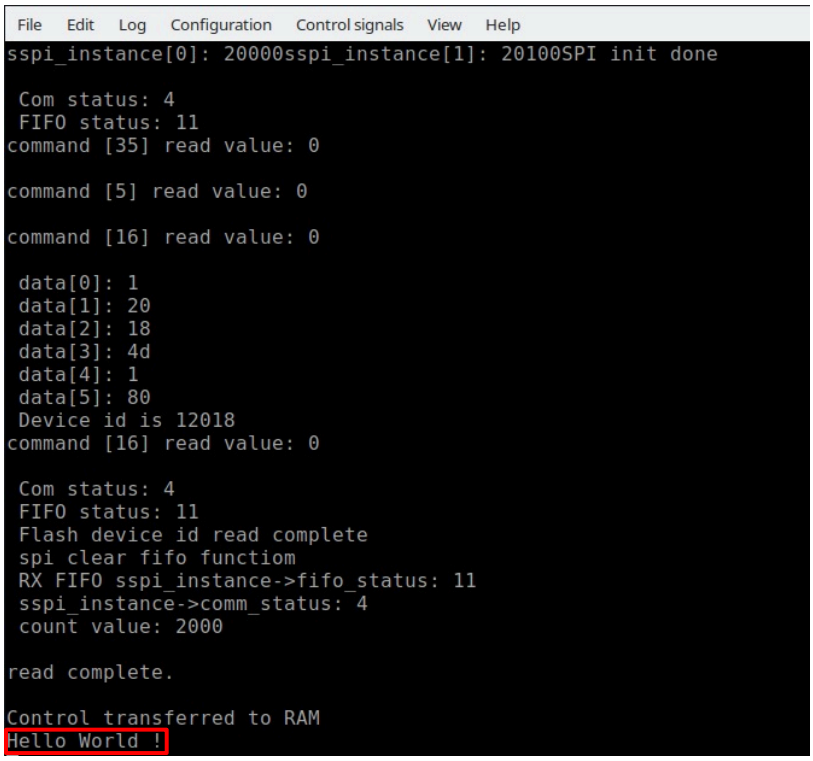


figure 38 : Application starts executing from the flash and output is visible in gtkterm